

Milestone 2: Perception Pipeline

Abdullah Amir (aa08453), Hamad Asif Khan (hk08394), Shaheer bin Tariq (st09213)

1. Task Definition

The **Intel RealSense depth camera** is mounted on top of the workspace, hence we assume that we are dealing with only the change in orientations about the z-axis as the faces of the cubes are nearly identical on all sides.

The camera is mounted in a "top-down" configuration with:

- **Origin (0,0,0):** The center of the camera lens.
- **Z-axis:** Points out from the plane of the cubes, down from the lens (depth).
- **X-axis:** Points from top left to top right of the plane
- **Y-axis:** Runs from top to bottom on the plane

We also assume that we are dealing with 4 main colors for the cubes: **Red, Green, Blue, Yellow.**

Our task is to identify the color, and position, and orientation with respect to the camera frame, given cubes of any, or all of these 4 colors.

2. Pipeline Design and Implementation

Our image processing pipeline begins by isolating the top layer of the boxes through plane-fitting at a specific height. We then segment the image using the following workflow:

Color Segmentation: We use K-means clustering to oversegment the image, then group clusters by matching them to a set of reference RGB values.

Instance Labeling: Each color group is treated as a binary mask. We apply watershed segmentation to separate overlapping blocks, creating distinct labels for every object.

Pose Estimation: For each labeled block, we calculate the major and minor axes using regionprops. The angle between the major axis and the x-axis provides the rotation (θ).

Localization: Since all rotations occur around the z-axis, we can easily construct a rotation matrix. The final 3D coordinates are retrieved by querying the point cloud at the center of each object's bounding box.

Find_plane.m

This is a wrapper around pcfitplane. It is responsible for removing the planes underneath the cubes in the workspace.

Segment.m

This just runs k means clustering with $k = 6$ in RGB space for the categories mentioned before.

mergeSegments.m

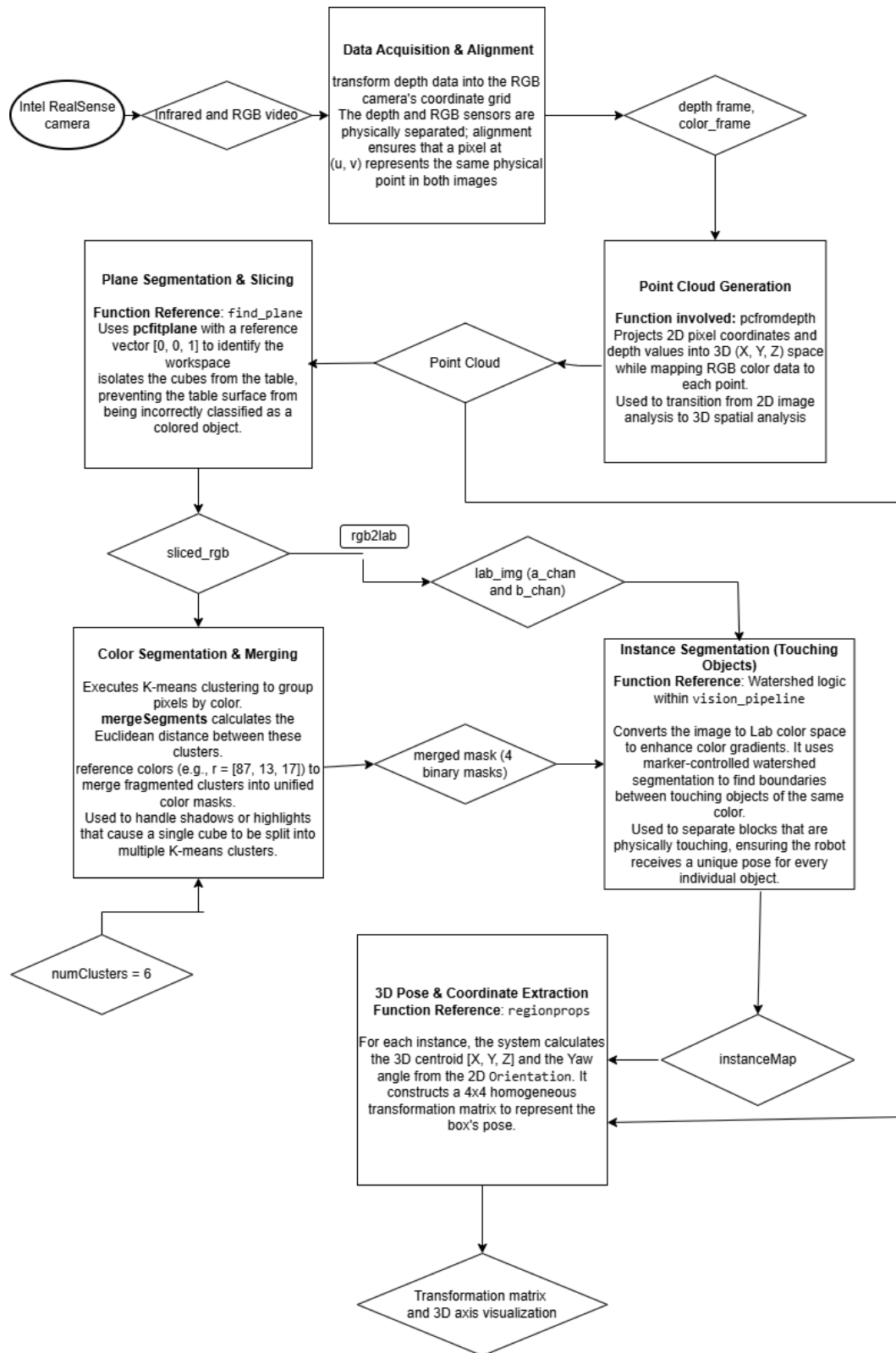
This compares each cluster to a certain known color and then reassigns it a single color label if they share the same color (handles over segmentation cases).

Vision_pipeline.m

This code generates visualizations. Calls the previous function in sequence and calculates the transformation matrix using orientation information obtained from the connected components in the cluster using regionprops, and position information from the point cloud.

Vision_pipeline_test.m

This is the major function where things are tested. It can be run in either “stream”, “file” or “test” mode. In test mode it will go through a variety of indexed point clouds, compare the labellings against what we as humans expect to see, and will automatically test whether our segmentation pipeline is appropriate or not.



3. Results

Test Setup

The test setup involved dealing with the workspace with placing cubes of

- different colors at
- random positions, longitudinally or laterally,
- With gaps, and without (next to each other)

The lighting conditions were also varied by collecting images of scenes with overhead light of the lab station on and off.

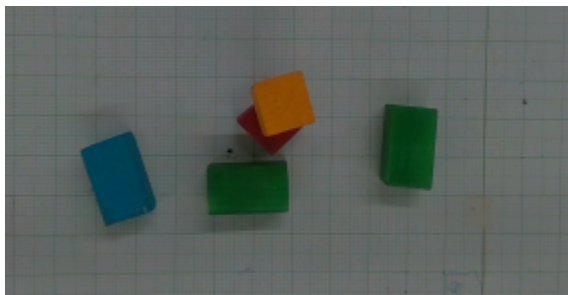
Test Details

The tests were performed in code with point cloud files for different test setups. It was automated to load each point cloud file in a loop and identify the color of the cube(s) in the scene and also output its correct homogeneous transformation matrix with respect to the camera frame.

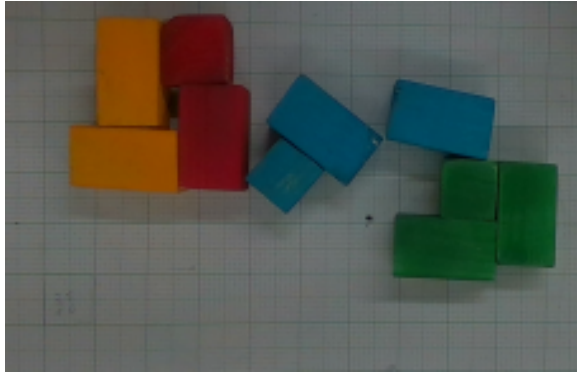
We collected three different datasets. One with 25 scenes, of purely rgb images. These included many edge cases, such as cubes on top of each other, cubes disparate, single cubes, multiple colored cubes, etc. Some characteristics are shown below:

Some characteristic images are as follows:

Cubes on top of each other:



Multiple cubes touching:



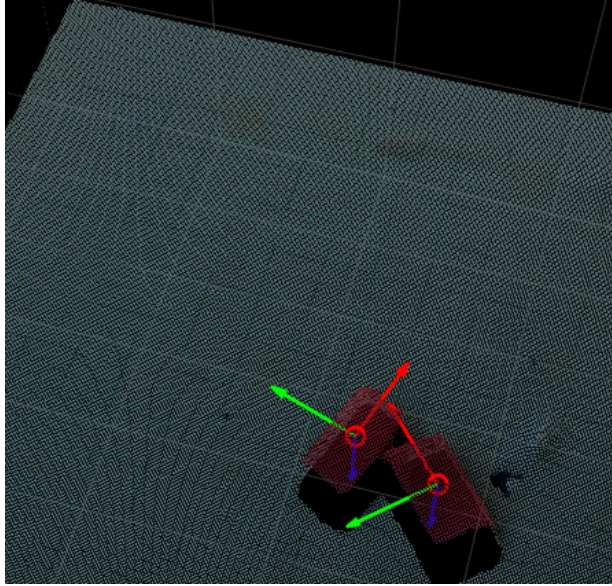
We then however decided a pure RGB pipeline would not be sufficient for our task. And so we shifted to pointclouds. We collected two different datasets. One with five images, and the other with 27 pointclouds.

We mainly developed our algorithms keeping the five images in mind, with the 27 images being used for further exploration. These included similar cases as the above RGB. In particular, the first one was a single cube, the second a touching cube, and the rest were stacked cubes. We later chose to discard the touching and stacked cubes situations for the labelling task, however we did use them to test our clustering/merging algorithm.

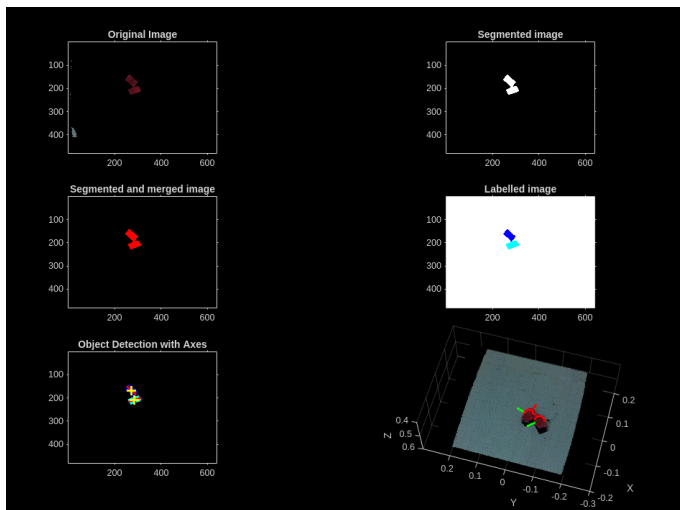
For this we created an expected output for each of the datasets, and then on each run of the whole project, we would run the files and test against the expected outputs. Our oversegment-then-merge algorithm performed very well, passing 5/5 test cases. Of course, we do plan to test this more rigorously with the other dataset we have collected.

```
read_flag =  
    "test"  
  
Test 1: [Desired: red          ] | [Recieved: red          ] ✓  
Test 2: [Desired: red          ] | [Recieved: red          ] ✓  
Test 3: [Desired: yellow       ] | [Recieved: yellow       ] ✓  
Test 4: [Desired: red, yellow  ] | [Recieved: yellow, red ] ✓  
Test 5: [Desired: blue         ] | [Recieved: blue         ] ✓  
>> ✱ Press [Ctrl] + [Shift] + [P] to generate code with Copilot
```

A successful complete run despite touching boxes results in this:



With each of the intermediate stages as

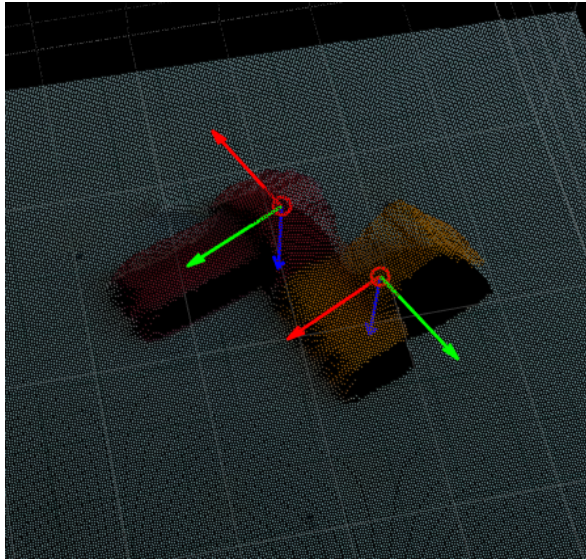


4. Failure Analysis

A major assumption that our pipeline rests on is the exclusion of all out of plane rotations - we only consider rotations about the z-axis, as it makes it easier for us to deal with objects and assume they are resting on a smooth surface with uniform elevation.

This assumption is likely not to cause any major issues as we expect the robot to pick and place cubes that will be spread around the workspace evenly on the plane, and not

cover each other. But if we consider a scenario like the robot is stacking cubes to make a tower or a pyramid, and the pile falls, it might not be able to pick and place all cubes correctly from there if the objects overlap and are oriented towards the camera with edges and multiple faces visible at once, as it is only programmed to detect the a major top face of the cuboid.



The failure can be observed here, in both assigning labels, and also from the subsequent bounding box detection and orientation calculations.

5. Next Steps

For increased robustness, the pipeline should be able to detect different objects of the same color in the same cluster, (which it is unable to do when the objects are stacked). This might involve experimenting with Edge Detection methods and Hough Transform, or exploring point cloud methods like ICP and point cloud registration.

Furthermore, we need to improve our testing setup. And so we're going to label more images, and try to test for certain specific, common orientations. I.e. 0 degrees and 45 degrees for the small cube, and 0 degrees, 45 degrees, 90 degrees, and 135 degrees for the small rectangle. This will give us confidence in our algorithms. Without this key insight we were worried that very random orientations wouldn't be detected at all, and so didn't create a labelled dataset for orientations.